

A 28 nm 16-kb Sign-Extension-Less Digital-Compute-in-Memory Macro With Extension-Friendly Compute Units and Accuracy-Adjustable Adder-Tree

Xin Si¹, Fangyuan Dong¹, Shengnan He, Yuhui Shi¹, Anran Yin, Hui Gao, and Xiang Li¹

Abstract—Conventional digital-domain SRAM compute-in-memory (CIM) faces challenges in handling multiply-and-accumulate (MAC) operations with signed values, either in serial data feeding mode or extra sign-bit processing. The proposed CIM macro has the following features: 1) a sign-extension-less array multiplication circuit structure that eliminates the need for converting partial sums into 2's complement, which removes the constraints related to handling specific symbol bits; 2) developing a circuit that avoids signed bit extension shift and accumulate, resulting in reduced area cost; and 3) integrating an adder structure that provides adjustable accuracy, thereby enhancing network adaptability as compared to traditional approximation techniques. A fabricated 28 nm 16-kb sign-extension-less DCIM was tested with the highest MAC speed with 5.6 ns (Signed 8 b IN&W 23 b Out) and achieved the best energy efficiency with 40.15 TOPS/W over a wide range of network adaptability.

Index Terms—Approximate calculations, artificial intelligence (AI), compute-in-memory (CIM), digital CIM, SRAM.

I. INTRODUCTION

With the continuous development of artificial intelligence (AI) technology and the wide application of cloud computing, the CNN model has achieved extraordinary results in many AI fields. At present, the CNN accelerator based on the von-Neumann architecture has been widely used in fields such as language translation. However, the von-Neumann architecture has limited its further development due to factors such as limited bus bandwidth. In order to break the von-Neumann bottleneck, the compute-in-memory (CIM) architecture was proposed. SRAM-based CIM has demonstrated superior energy efficiency compared to von-Neumann architecture when it comes to matrix multiplication. In recent times, the implementation of CIM can be broadly categorized into analog and digital modes. However, the analog-domain SRAM CIM [1], [2], [3], [4], [5], [6], [7] is susceptible to process variations and constrained by signal margins. Digital-domain SRAM CIM exhibits superior robustness compared to analog-domain CIM, making it advantageous in high-precision and high-throughput application scenarios. However, conventional digital CIM [8], [9], [10], [11], [12], [13], [14], [15] faces several notable challenges as follows:

- 1) The traditional multiplication algorithm requires converting the final partial sum to 2's complement, resulting in an additional multiply-and-accumulate (MAC) cycle for the multiplication of signed IN and W. Additionally, the feature-wise shift and accumulation operations add to the area requirements for symbol bit evaluation.

- 2) The large parallel adder tree introduces significant power consumption.
- 3) The lack of MAC precision control limits its ability to achieve optimal power, performance, and area (PPA) trade-offs, thereby restricting its effectiveness to a limited range of network configurations.

Significant advancements have been made to address these challenges. One improvement is the implementation of a sign-extension-less array multiplication circuit, eliminating the need for converting partial sums into 2's complement representation and removing constraints related to handling specific symbol bits. Another development is a circuit that avoids signed bit extension shift and accumulation, reducing area costs. Additionally, low-power full adders and an integrated adder structure with adjustable accuracy have been introduced in the parallel adder tree. This enhances the adaptability of the network while reducing the power consumption of the adder tree compared to traditional designs. A fabricated 28 nm 16-kb Sign-extension-less DCIM was extensively tested, achieving the highest MAC speed of 5.6 ns (with signed 8-bit input and weight, and 23-bit output). Moreover, it demonstrated remarkable energy efficiency, achieving 40.15 TOPS/W across a wide range of network adaptability scenarios, including traditional deep convolution and large kernel size convolution.

This brief is divided into five sections, the first of which is the Introduction. Section II presents the principle and data flow of our sign-extension-less CIM. Section III describes the detailed architecture of the proposed CIM macro. Design implementation and its simulation results are discussed in Section IV and Section V concludes this brief.

II. SIGN-EXTENSION-LESS CIM PRINCIPLE

A. CIM Data Flow

When performing 8 b signed MAC operations, we first divide the input 8 b signed numbers into four groups of 2 b numbers. These four groups of 2 b input are then fed into the array along with the weight for MAC over four cycles. During the MAC process, we encounter four distinct scenarios as follows: In the first scenario, we have a 2 b input (including the sign bit) multiplied by a 1 b signed weight. The MSB in the 2 b outputs remains unaltered, while the LSB is inverted. In the second scenario, a 2 b input (including the sign bit) is multiplied by a 1 b unsigned weight. Here, the MSB in the 2 b outputs is inverted, while the LSB remains unaltered. In the third scenario, we consider a 2 b input (excluding the sign bit) multiplied by a 1 b signed weight, resulting in both 2 b outputs being inverted. Last, in the fourth scenario, a 2 b input (excluding the sign bit) is multiplied by a 1 b unsigned weight, and both 2 b outputs remain unaltered.

Fig. 1(a) demonstrates the operation process using an example of MAC with two 4 b signed numbers. According to our method, completing a set of 4 b signed MAC requires only two cycles, reducing the computation period by half compared to previous works [8], [9], [10].

Manuscript received 1 March 2024; revised 23 April 2024 and 28 May 2024; accepted 17 June 2024. Date of publication 3 July 2024; date of current version 24 October 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2022ZD0118902 and in part by the National Natural Science Foundation of China under Grant 92264203 and Grant 62204036. (Corresponding author: Xiang Li.)

The authors are with the School of Integrated Circuits, Southeast University, Nanjing 211189, China (e-mail: 363393606@qq.com).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TVLSI.2024.3418888>.

Digital Object Identifier 10.1109/TVLSI.2024.3418888

1063-8210 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

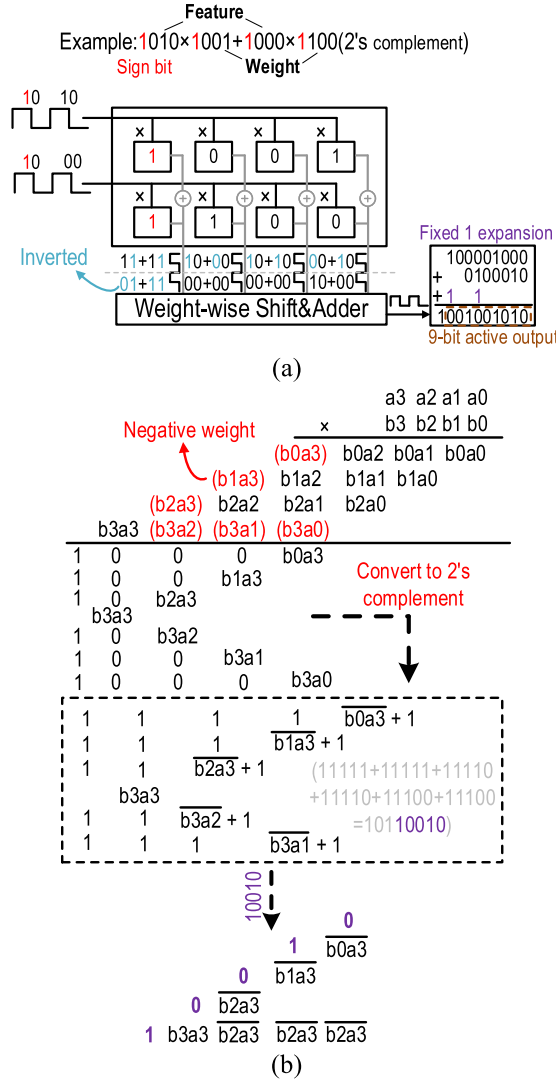


Fig. 1. (a) Example of a data flow for our sign-extension-less CIM. (b) Mathematical principle of sign-extension-less multiplication algorithms.

B. Mathematical Principle

The mathematical principles of the proposed macro with fixed “1” complement operation as shown in Fig. 1(b). Taking 4 b signed number multiplication as an example, when performing multiplication with the two 4 b numbers, typically four partial sums are generated. Among these partial sums, the results representing the multiplication of 1 b signed bits and 1 b unsigned bits have negative bit weights. By converting these negative bits into 2’s complement form and simplifying, the process of sign bit alignment in the original partial sums and the conversion of the last partial sum into 2’s complement can be simplified to bitwise inverse and fixed “1” complement operations.

III. PROPOSED CIM MACRO

A. Overall Architecture

Fig. 2 illustrates the architecture of our sign-extension-less SRAM CIM macro, which incorporates a compact 6 Ts plus unsigned/signed-weight compute unit (USCU/SCU). The macro consists of a 128×128 b compact 6 T SRAM array, a 128×2 b input buffer, accuracy level control (ALC), input symbol bit flag (SIGN), WL driver, and R/W IO. The CIM array, with dimensions of 128×128 b, is divided into a single bank dedicate to storing the signed bits of

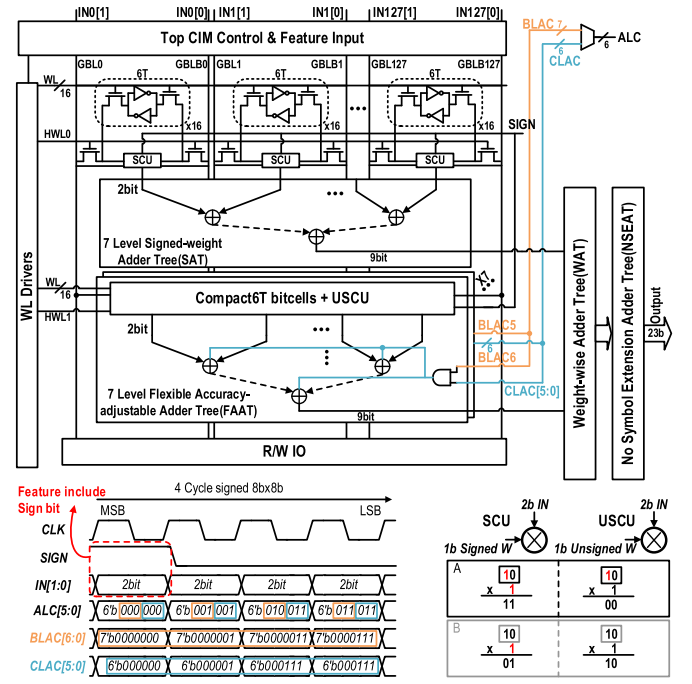


Fig. 2. Proposed sign-extension-less SRAM CIM macro structure and the real data flow.

8 b weights and seven banks responsible for storing the remaining bits of 8 b weights. The signed-weight bank is composed of 16×128 b compact 6 Ts, 128 SCUs, and a seven-level signed-weight adder tree (SAT). Each unsigned-weight bank consists of 16×128 b compact 6 Ts, 128 USCUs, and a seven-level flexible accuracy-adjustable adder tree (FAAT). The macro operates in two distinct modes: memory and compute. In the memory mode, it functions similar to a traditional SRAM macro, enabling the storage of weight data in each bank through R/W IO operations. In compute mode, the macro performs 128 signed INT8 IN and INT8 W MAC operations. First, the 8-bit W data is written into the array using memory mode. Then, the 8-bit IN data is inputted into the macro through the feature-in buffer over four cycles. During each cycle, the 1-bit SIGN and 6-bit ALC signals are transmitted to the macro, facilitating the inverse operation on the intermediate data and controlling the level of accuracy. Ultimately, a 23-bit signed INT MAC value is obtained ($\sum(\text{IN}[(7:0)] \times \text{W}[(7:0)])$). The ALC can be configured to adjust the accuracy state in each cycle. Its upper 3 bits are decoded to generate a 7-bit bank-level accuracy control signal (BLAC), while the lower 3 bits are also decoded to generate a 6-bit channel-level accuracy control signal (CLAC). In full precision mode, all 6 bits of the ALC are set to “0,” resulting in BLAC and CLAC both being “0.” In approximate mode, the signal obtained by performing a logical AND operation between BLAC and CLAC controls the activation of approximate adders within the adder tree. By configuring different values for the ALC, different levels of approximation effects can be achieved.

B. Local Computing Circuit

In Section II, we simplified the process of calculating 2’s complement and sign extension in signed multiplication by using bitwise negation and fixed “1” complementation. In this section, we propose a local computation circuit (SCU/USCU) for implementing the four types of bitwise negation (shown in Fig. 3). Following the principles of the multiplication algorithm, we further divide the operation of $2 \text{ b IN} \times 1 \text{ b W}$ into four cases of $1 \text{ b IN} \times 1 \text{ b W}$: “signed

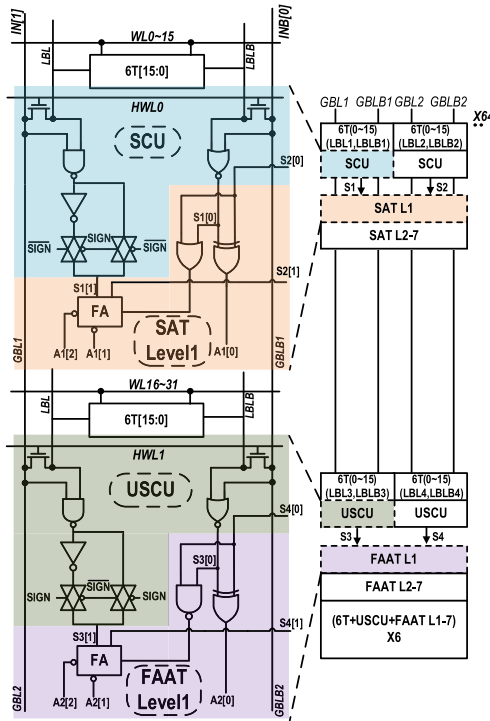


Fig. 3. Local computing unit and the first level adder tree.

W $\times$ signed IN, No inverse,” “signed W $\times$ unsigned IN, inverse,” “unsigned W $\times$ signed IN, inverse,” and “unsigned W $\times$ unsigned IN, No inverse.” Therefore, two separate compute units have been devised to accommodate the two weight cases (signed/unsigned). Each compute unit integrates a control signal denoted as “SIGN” to regulate the presence of sign bits in the 2 b input. When sign bits are present, the signal “SIGN” is set to 1; otherwise, it is negated. To optimize transistor utilization, the inverter within the multiplication compute unit is integrated into the first-level addition tree in the case of “unsigned W $\times$ signed IN, inverse,” effectively reducing the number of required inverters by 50%.

C. Adder Tree

In previous CIM works, the inference tasks were typically limited to fixed precision, such as most analog works [2], [3], [4], [5] and some digital works [11]. These approaches often suffered from poor network adaptability due to the presence of approximation errors in the output. On the other hand, works that achieved full precision outputs [8], [9], [10] incurred higher power consumption when facing inference tasks that did not require such high precision. Therefore, in this brief, we propose an accuracy-adjustable adder tree to achieve a trade-off between PPA. The specific structure of the proposed structure is illustrated in Fig. 4. The ALC signal, after being decoded, controls the approximate activation state of the FFAs in the adder tree.

To optimize area utilization and minimize power consumption, two types of transmission function full adders, (M1 and M2) [16] are employed in the adder tree [as shown in Fig. 5(a)]. Inverters are incorporated to enhance the driving capability for subsequent stages when cascaded. For controllable accuracy, a flexible full adder (FFA) circuit [as shown in Fig. 5(b)] based on the 22 T full adder is devised. In approximate mode, the FFA is simplified to a 14 T complementary CMOS approximate full adder. In this mode, the carry output remains unchanged, while the sum output becomes:

$$\overline{\text{Sum}} = \text{Co.} \quad (1)$$

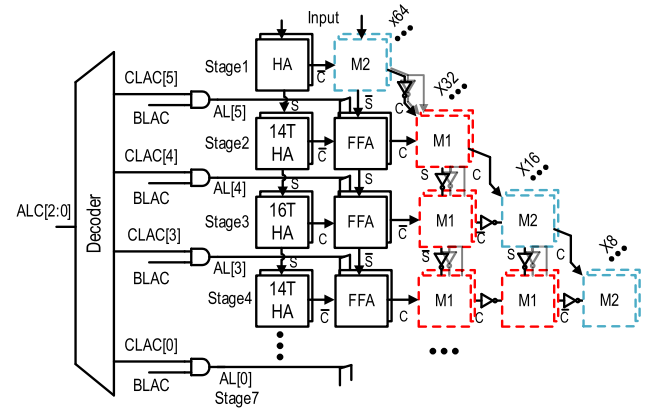


Fig. 4. Architecture of the FAAT.

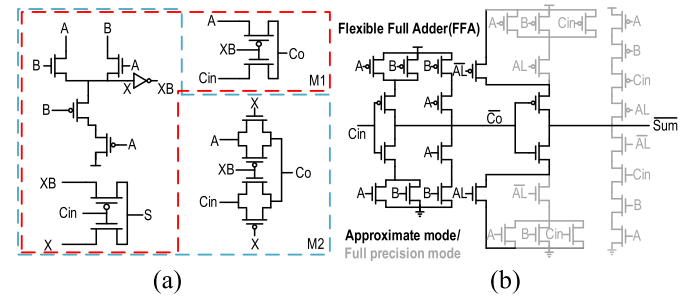


Fig. 5. (a) 14 T (M1) and 16 T (M2) transmission function full adders. (b) FFA circuit.

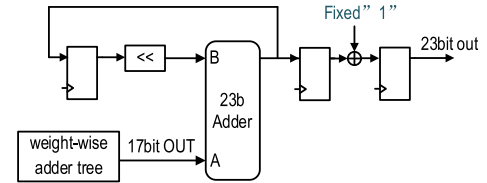


Fig. 6. Signed bit extension-less shift and accumulate.

FFAs are exclusively deployed for low-order bit additions within the adder tree to avoid an exponential amplification of approximation errors. Each level of the adder tree is governed by an accuracy control signal (AL), decoded by the ALCs. Each FAAT offers seven configuration modes, including the full-precision mode. Through experimentation, it has been observed that when our FAATs operate in minimum accuracy mode, they can save approximately 20% of power consumption compared to the full precision mode. Furthermore, in comparison to the conventional adder tree, our FAATs in minimum accuracy mode can still achieve maximum power savings of around 13%. Following the accumulation of the addition output result through the weight-wise adder tree, the feature-wise shift accumulation is completed over four cycles. The feature-wise shift accumulation (as shown in Fig. 6) does not consider the sign bits, and the final output result of 23 bits is obtained after adding “1” to the corresponding position.

IV. DESIGN IMPLEMENTATION AND EVALUATION

A. Design Implementation

Fig. 7 illustrates the layout photograph, area breakdown, and performance summary of the proposed MAC macro. The macro was implemented in 28 nm CMOS technology, occupying 0.036 mm² of space and incorporating 16 kb SRAM memory. The access time was measured at 6.4 ns in the full-precision mode at 0.9 V. However, when

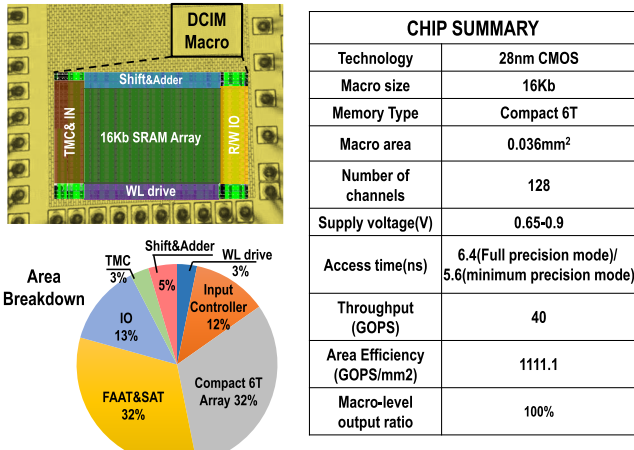


Fig. 7. Layout photograph, area breakdown, and performance summary.

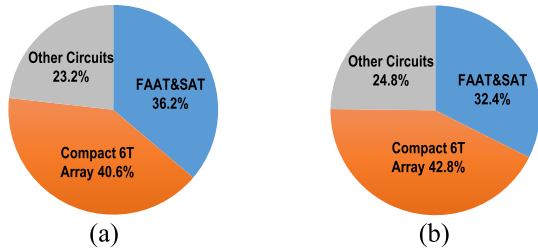


Fig. 8. Energy breakdown. (a) Full accuracy mode. (b) Minimum accuracy mode.

the approximation mode is enabled, the access time can even reach as low as 5.6 ns. This brief achieves the highest area efficiency of 1.11 TOPS/mm² at a voltage of 900 mV and a frequency of 625 MHz. Notably, the compact 6 T design of our macro ensures that SRAM occupies only 32% of the total area.

Fig. 8 shows the energy breakdown in two modes. Our design's parallel adder tree only occupies 36.2% of the overall power consumption in full-precision mode. Enabling the approximate mode will further reduce power consumption.

B. Evaluation of Approximation

We evaluated the inference accuracy of ResNet50@Cifar100 at different levels of approximation by configuring different ALC values during the four cycles of 8-bit MAC operations in the convolutional layer. From Fig. 9, it can be observed that by manipulating different ALC values, we are able to achieve fine-grained control over the network accuracy. This allows us to implement different precision modes for computation to meet the requirements of different application scenarios.

C. Performance Simulation Results

Fig. 10 shows the performance results of our CIM macro. With a power supply of 0.9 V, our proposed CIM macro achieves a maximum operating speed of 714 M in the minimum accuracy and full accuracy modes, respectively. In the worst-case scenario (8-bit input, 8-bit weight, 23-bit output, 0.65 V, and 10% sparsity in input and weight), we achieved the highest efficiency of 26.49 TOPS/W in the minimum accuracy mode and 21.87 TOPS/W in the full-precision mode. Additionally, with increasing sparsity of inputs and weights, we can attain even higher energy efficiency, reaching up to 33.58 TOPS/W (IN = 8, W = 8 b, Out = 23 b, full precision) and

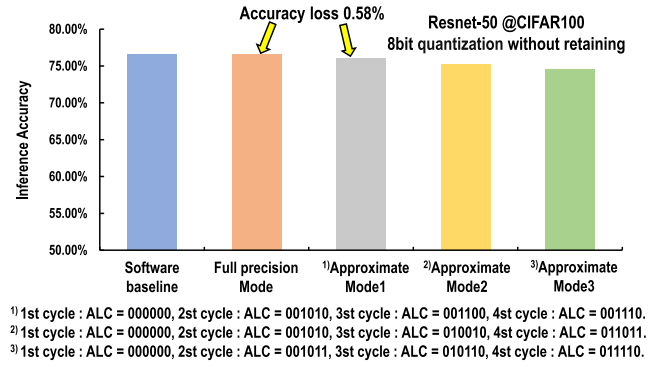


Fig. 9. Evaluation results of inference accuracy on ResNet-50@Cifar100.

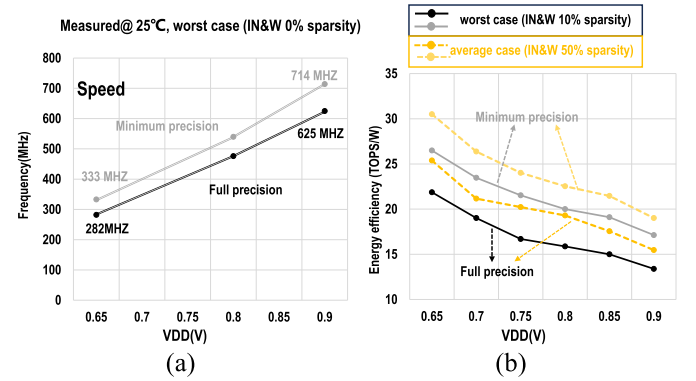


Fig. 10. Simulation results. (a) Frequency with different VDD. (b) Energy efficiency with different VDD cases.

TABLE I
MACRO PERFORMANCE COMPARISON

	ISSCC'23 [1]	ISSCC'22 [8]	ISSCC'21 [10]	This work
Technology (nm)	22	28	28	28
Array size	128Kb	32Kb	64Kb	16Kb
Macro size	0.42	0.03	0.202	0.036
MAC Operation	Analog	Digital	Digital	Digital
Supply Voltage(V)	0.72-0.82	0.8	0.72	0.65-0.9
Compute functionality	Full precision	Full precision	Full precision	Accuracy adjustable
Memory density (Kb/mm ²)	304.76	1067	316.8	443.82
Input precision	8	8	8	8b
Weight precision	8	8	8	8b
Output precision	24	21	24	23b
MAC cycle (Signed 8bit)	8	8	8	4
Access time (ns)	22.16	24	18	5.6
Area efficiency (GOPS/mm ²)	1428	193	4539	1111
Energy Efficiency (TOPS/W)	21.38	27.38	24.7	13.31-40.15(8b)

40.15 TOPS/W (IN = 8 b, W = 8 b, Out = 23 b, minimum accuracy mode).

Table I shows that analog-domain CIM techniques have achieved high throughput with a 100% output ratio in recent years. However, they suffer from high power consumption and longer processing time when dealing with 8-bit MAC operations. On the other hand, digital-domain CIM using single-bit serial input for data limits the circuit's operating speed. This study focuses on mitigating the additional

MAC cycle overhead caused by sign-bit processing. Additionally, the proposed accuracy-adjustable adder circuit offers an improved trade-off between network performance and circuit performance.

V. CONCLUSION

In this brief, we propose a novel implementation approach for CIM. This CIM macro enables the support of 128 8 b IN and 8 b W MAC operations with arbitrary precision control within four cycles. It achieves a fast 8 b access time of 5.6 ns and an energy efficiency of 40.15 TOPS/W, along with a wide range of network adaptability.

REFERENCES

- [1] P. Chen et al., "7.8 A 22 nm delta-sigma computing-in-memory ($\Delta\Sigma$ CIM) SRAM macro with near-zero-mean outputs and LSB-first ADCs achieving 21.38 TOPS/W for 8 b-MAC edge AI processing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 140–142.
- [2] X. Si et al., "A local computing cell and 6T SRAM-based computing-in-memory macro with 8-b MAC operation for edge AI chips," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, Sep. 2021.
- [3] J. Kim and J. Park, "A charge-domain 10T SRAM based in-memory-computing macro for low energy and highly accurate DNN inference," in *Proc. 18th Int. SoC Design Conf. (ISOCC)*, Oct. 2021, pp. 89–90.
- [4] X. Si et al., "A twin-8T SRAM computation-in-memory unit-macro for multibit CNN-based AI edge processors," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 189–202, Jan. 2020.
- [5] B. Wang et al., "SNNIM: A 10T-SRAM based spiking-neural-network-in-memory architecture with capacitance computation," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Austin, TX, USA, May 2022, pp. 3383–3387.
- [6] M. Gong, N. Cao, M. Chang, and A. Raychowdhury, "A 65 nm thermometer-encoded time/charge-based compute-in-memory neural network accelerator at 0.735 pJ/MAC and 0.41 pJ/update," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 4, pp. 1408–1412, Apr. 2021.
- [7] B. Wang et al., "A 28 nm horizontal-weight-shift and vertical-feature-shift-based separate-WL 6T-SRAM computation-in-memory unit-macro for edge depthwise neural-networks," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, Feb. 2023, pp. 134–136.
- [8] B. Yan et al., "A 1.041-Mb/mm² 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28 nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 188–190.
- [9] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.
- [10] H. Mori et al., "A 4 nm 6163-TOPS/W/b 4790-TOPS/mm²/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 132–134.
- [11] A. Guo et al., "A 28 nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 128–130.
- [12] H. Kim, T. Yoo, T. T. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.
- [13] Y.-D. Chih et al., "16.4 An 89 TOPS/W and 16.3 TOPS/mm² all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 252–254.
- [14] C. Zhang, M. Wang, Y. Mai, C. Tang, and Z. Yu, "A high-density and reconfigurable SRAM-based digital compute-in-memory macro for low-power AI chips," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3589–3593, Sep. 2023.
- [15] H. Kim, J. Mu, C. Yu, T. T. Kim, and B. Kim, "A 1–16 b reconfigurable 80 Kb 7T SRAM-based digital near-memory computing macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 4, pp. 1580–1590, Apr. 2023.
- [16] J.-F. Lin, Y.-T. Hwang, M.-H. Sheu, and C.-C. Ho, "A novel high-speed and energy efficient 10-transistor full adder design," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 54, no. 5, pp. 1050–1059, May 2007.